

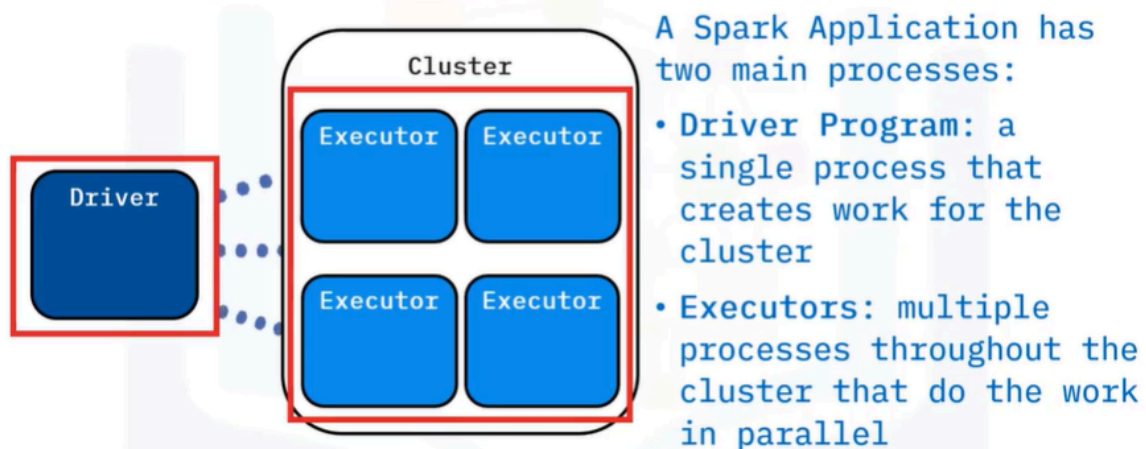
Apache Spark Architecture

Sure! Let's dive into the concept of **Spark Jobs and Tasks** in simple terms.

In Apache Spark, a **Job** is like a big project you want to complete, such as baking a cake. When you decide to bake a cake, you have a list of tasks to do, like mixing the ingredients, baking it in the oven, and decorating it. Each of these smaller tasks is what we call a **Task** in Spark. So, when you run a Spark application, it creates a Job, which is then broken down into smaller Tasks that can be done simultaneously, just like you can mix ingredients while the cake is baking.

To visualize this, think of a team of chefs in a kitchen. Each chef can work on a different task at the same time, making the whole process faster. In Spark, multiple Tasks can run in parallel across different machines (or Executors), allowing for efficient processing of large datasets. This parallelism is what makes Spark powerful!

Spark Application Processes



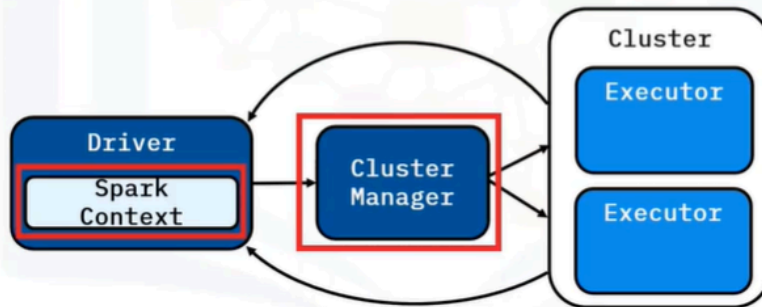
- A Spark application has two main processes. The driver program runs as one process per application. The driver process can be run on a cluster node or another machine as a client to the cluster. The driver runs the application's

user code, creates work and sends it to the cluster. An executor is a process running multiple threads to perform work concurrently for the cluster. Executors work independently. There can be many throughout a cluster and one or more per node, depending on configuration.

What is the Spark Context

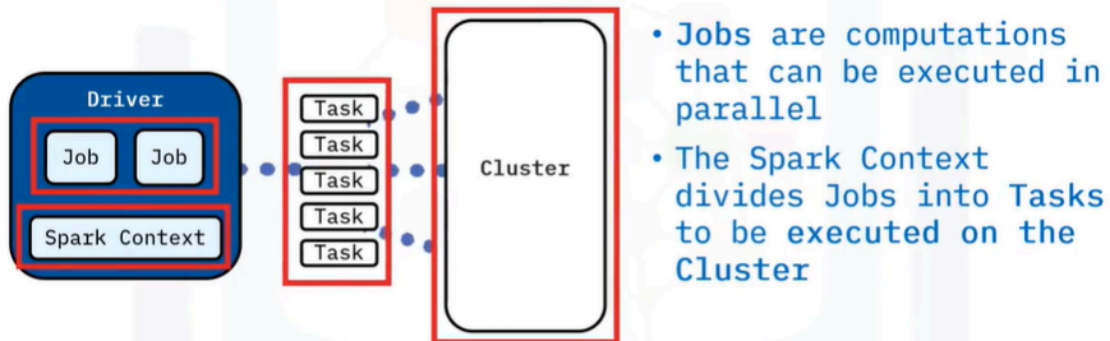
Spark Context:

- Communicates with the Cluster Manager
- Is defined in the Driver, with one Spark Context per Spark Application



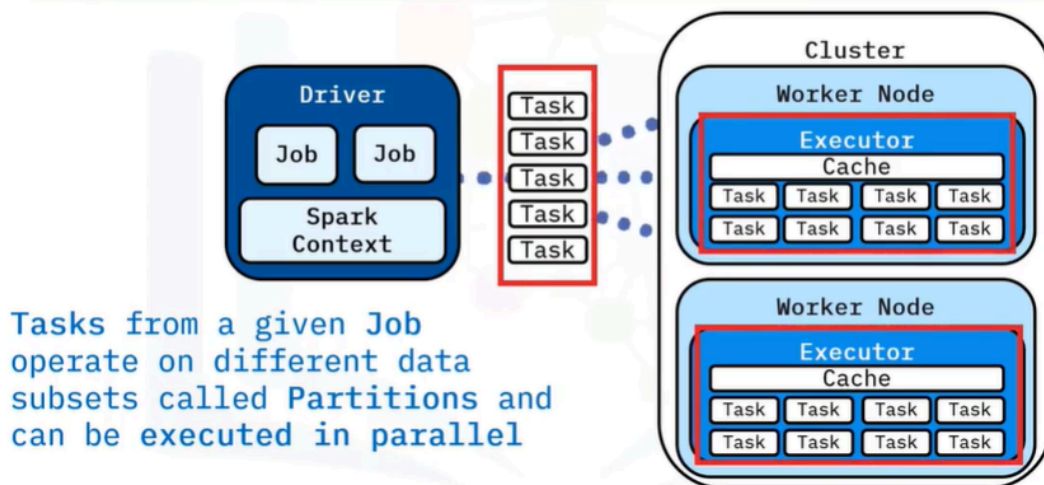
- The Spark Context starts when the application launches and must be created in the driver before DataFrames or RDDs. Any DataFrames or RDDs created under the context are tied to it and the context must remain active for the life of them.

Spark Jobs



- The driver program creates work from the user code called "Jobs" (or computations that can be performed in parallel). The Spark Context in the driver divides the jobs into tasks to be executed on the cluster.

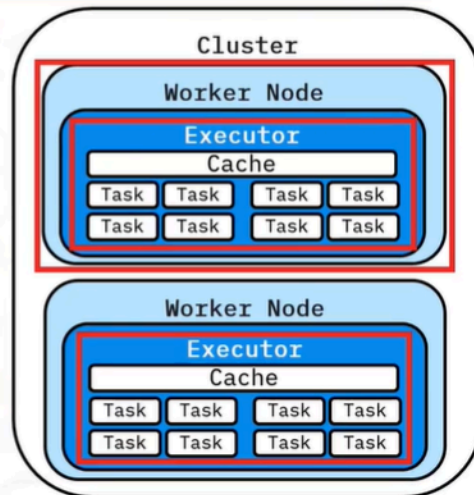
Spark Tasks



- Tasks from a given job operate on different data subsets, called Partitions. This means tasks can run in parallel in the Executors.

Spark Executors run tasks from a job

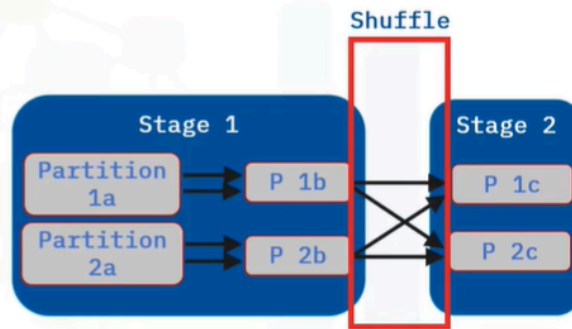
- A Worker is a cluster node that can launch executor processes to run tasks
- Each Executor is allotted a set number of cores that each run one task at a time
- Increasing Executors and cores increases cluster parallelism
- Ideally, limit Executor x core combinations to total cores per node – the example here shows 1 x 8



- A Spark Worker is a cluster node that performs work. A Spark Executor utilizes a set portion of local resources as memory and compute cores, running one task per available core. Each executor manages its data caching as dictated by the driver. In general, increasing executors and available cores increases the cluster's parallelism. Tasks run in separate threads until all cores are used. When a task finishes, the executor puts the results in a new RDD partition or transfers them back to the driver. Ideally, limit utilized cores to total cores available per node. For instance, an 8-core node could have 1 executor process using 8 cores.

Spark Stages and Shuffles

- A Stage is a set of tasks within a job that can be completed on the current local data partition
- A Shuffle marks the boundary between Stages
- Stages connect to form a dependency graph



- A "stage" in a Spark job represents a set of tasks an executor can complete on the current data partition. When a task requires other data partitions, Spark must perform a "shuffle." A shuffle marks the boundary between stages. Subsequent tasks in later stages must wait for that stage to be completed before beginning execution, creating a dependency from one stage to the next.

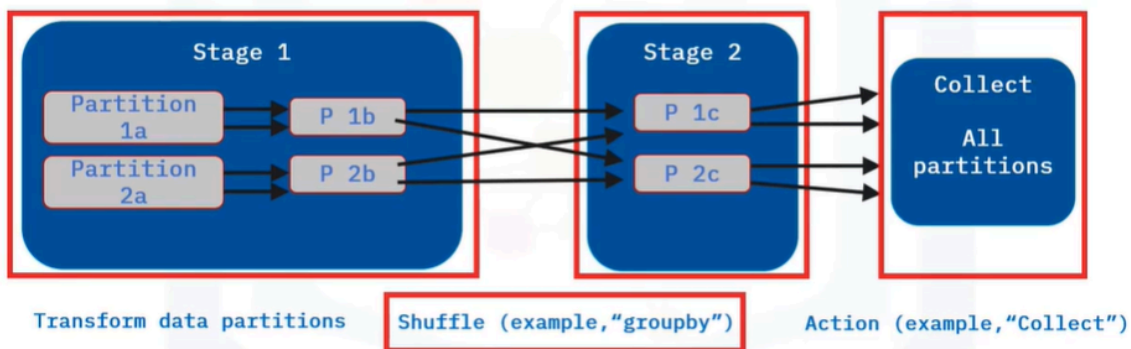
Why Shuffle Data

A Shuffle is:

- Costly – requiring data serialization, disk and network I/O
- Necessary when an operation requires data outside the current partition of the task
- How Spark re-distributes the dataset across the cluster

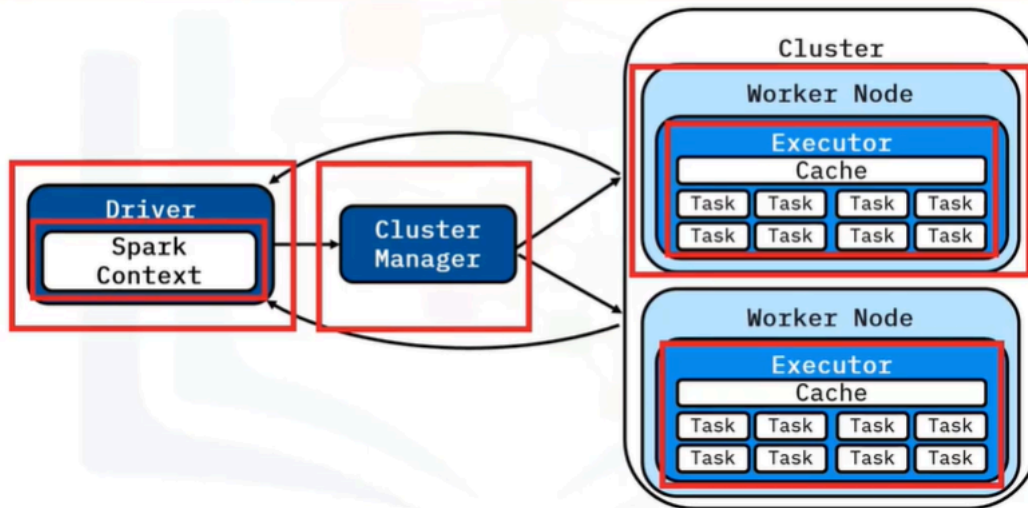
- Shuffles are costly as they require data serialization, disk and network I/O. This is because they enable tasks to “pass over” other dataset partitions outside the current partition. An example would be a “groupby” with a given key that requires scanning each partition to find matching records. When Spark performs a shuffle, it redistributes the dataset across the cluster.

Shuffle Example



- This example shows two stages separated by a shuffle. In Stage 1, a transformation (such as a map) is applied on dataset “a” which has 2 partitions (“1a” and “2b”). This creates data set “b”. The next operation requires a shuffle (such as a “groupby”). Key values could exist in any other partition, so to group keys of equal value together, tasks must scan each partition to pick out the matching records.
- Transformation results are placed in Stage 2. Here results have the same number of partitions, but this depends on the operation. Final results are sent to the driver program as an action, such as collect. NOTE: It is not advised to perform a collection to the driver on a large data set as it could easily use up the driver process memory. If the data set is large, apply a reduction before collection.

Apache Spark Architecture

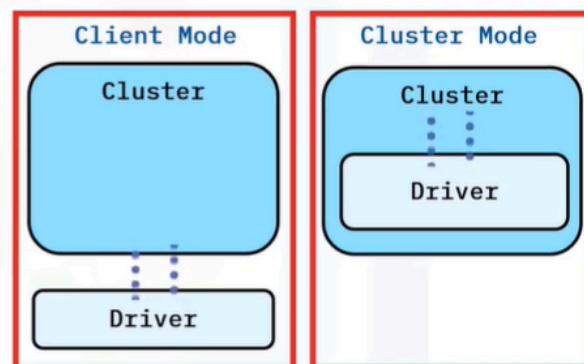


- Let's recap what we've learned about Spark Architecture. It consists of the driver and the executor processes. The cluster comprises the Cluster Manager and worker nodes. The Spark Context schedules tasks to the cluster and the Cluster Manager manages the cluster's resources.

Driver Deploy Modes

There are two deploy modes:

- **Client Mode** - the application submitter launches the driver process outside the cluster
- **Cluster Mode** - the framework launches the driver process inside the cluster



- The driver program can be run in either client or cluster mode. In client mode the application submitter (such as a user machine terminal) launches the driver outside the cluster. In cluster mode, the driver program is sent to and run on an available Worker node inside the cluster. The driver must be able to communicate with the cluster while it is running, whether it is in client or cluster mode.

Summary

In this video, you learned that:

- Spark Architecture has **driver** and **executor** processes, coordinated by the **Spark Context** in the **Driver**
- The **Driver** creates **jobs** and the **Spark Context** splits them into **tasks** which can be **run in parallel** in the **Executors** on the **Cluster**
- **Stages** are a set of tasks that are separated by a **Data Shuffle**
- **Shuffles** are **costly**, as they require data serialization, disk and network I/O
- The **Driver** can be run in either **Client Mode** (connecting the **Driver** outside the **Cluster**) or **Cluster Mode** (running the **Driver** in the **Cluster**)